Türk İşaret Dili Tanıma Projesi (Mobil & Web)

Düşük Sistem Gereksinimli Mobil Model (Ultra-Hafif TFLite), Web Uygulaması ve Canlıya Alma Yol Haritası

1. Telefonda ve Web'de Çalışacak Hafif Model Karşılaştırması

Uygulamanın mobil telefonlarda ısınma, pil tüketimi ve kasma yapmadan çalışabilmesi için **görüntü (piksel) tabanlı ağır modeller yerine koordinat (landmark) tabanlı ultra-hafif TFLite modeli** kullanılmalıdır:

Model Türü	Boyut	CPU / Pil Yükü	Telefonda Çalışır mı?	Değerlendirme
Mevcut `model.p` (Random Forest - 200 Ağaç)	**25.5 MB**	Orta (Sadece PC/Sunucu)	**HAYIR** (Scikit-Learn)	Mobil cihaza gömülemez; sürekli Python sunucusuna fotoğraf göndermeyi zorunlu kılar.
Görüntü Tabanlı CNN (`model_cnn.tflite`)	**2.75 MB**	**YÜKSEK** (Piksel İşleme)	**ZOR** (Telefonu Isıtır)	Her karede yüz binlerce pikseli işler; arka plan ve ışık değişiminde doğruluğu hızla düşer.
ÖNERİLEN: Landmark MLP (`assets/model.tflite`)	**~50 KB** *(500x Küçük)*	**ÇOK DÜŞÜK** (< 1 ms Tahmin)	**EVET** (Mobil + Web + PC)	Sadece MediaPipe'tan çıkan **84 sayısal koordinatı** işler. En eski telefonda bile 60 FPS çalışır.

> **Temel Mimari Avantajı:** Kameradan alınan görüntüde MediaPipe Hands ile 21 eklem noktası (landmark) bulunur ve sadece bu **84 adet sayı (float)** 3 katmanlı küçük bir sinir ağına (`model.tflite` — ~50 KB) verilir. Böylece model arka plandan, kıyafetten veya ışıktan hiç etkilenmez ve telefonda **0.2 milisaniyede** sonuç üretir.

2. Mevcut Kodda Tespit Edilen Kritik Sorunlar

Öncelik	Dosya	Sorun	Çözüm
KRİTİK	`api_socket.py` (Satır 218)	`x_ = []` tanımlanmadığı için el tespit edildiğinde `NameError` veriyor.	`data_aux = []` altına `x_ = []` eklenmeli.
KRİTİK	`2 veri işleme1.py`	El kameraya yaklaşıp uzaklaştığında ölçek değişiyor (sadece `x - min(x_)` var).	`(x - min(x_)) / (max(x_) - min(x_))` ile `[0, 1]` aralığına normalize edilmeli.
YÜKSEK	`3 model eğitimi...py`	Tek el (42) ve çift el (84) için iki ayrı model var; mobilde yönetimi zor.	Tek el verisine 42 sıfır (zero-padding) eklenerek **84 girdili tek bir TFLite model** eğitilmeli.
YÜKSEK	`recognition_screen.dart`	Ana UI thread üzerinde piksel piksel YUV420 → RGB dönüşümü telefonu yoruyor.	Cihaz içi MediaPipe + TFLite kullanılmalı veya dönüşüm `Isolate` içine alınmalı.

3. Dört Fazlı Geliştirme Yol Haritası

Faz 1: Veri Normalizasyonu & Hata Düzeltme

- `api_socket.py` içindeki `x_ = []` hatasının düzeltilmesi
- Ölçekten bağımsız `[0, 1]` normalizasyonu
- Tek el (42) + İki el (84) verilerinin 84 özellikte birleştirilmesi

Faz 2: Ultra-Hafif Mobil Model (TFLite) Eğitimi

- 3 katmanlı Keras MLP modelinin eğitilmesi (~21.000 parametre)
- Float16 Quantization ile **~50 KB** `model.tflite` üretimi
- `mobil_uygulama/assets/model.tflite` içine entegrasyon

Faz 3: Mobil (Offline/Online) & Web Uygulaması

- Mobilde `tflite_flutter` ile cihaz içi (internetsiz) hızlı tahmin
- Web tarayıcısı için kamera ve MediaPipe JS / WebSocket uyumu
- Mobil ve masaüstü ekranlara duyarlı (Responsive) arayüz

Faz 4: Canlıya Alma (Deployment)

- Android Release APK / Play Store paketlemesi
- Flutter Web çıktısının Vercel / Firebase Hosting'e yüklenmesi
- Python sunucusunun Docker ile Render / Railway'e taşınması

4. Rapor 1: Telefonda Çalışacak Hafif TFLite Modeli Nasıl Eğitilir?

4.1. Tek ve Çift El Verisini 84 Girdide Birleştirme (Zero-Padding)

Mobilde tek bir `model.tflite` dosyası kullanmak için girdi boyutu sabit **84 float** yapılır. Tek el algılandığında ilk 42 eleman elin koordinatları, son 42 eleman ise `0.0` olarak doldurulur. İki el algılandığında 84 elemanın tamamı doldurulur.

4.2. Keras ile Ultra-Hafif MLP Eğitimi ve TFLite Dönüşümü

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(84,)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(29, activation='softmax')  # 29 Türkçe Harf
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=50, batch_size=32)

# Mobil için sıkıştırılmış TFLite dönüşümü (~50 KB)
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()
open("mobil_uygulama/assets/model.tflite", "wb").write(tflite_model)
```

5. Rapor 2: Mobil ve Web Çalışma Mimarisi

Platform	Nasıl Çalışır?	Performans & Avantaj
Mobil (Android / iOS) Cihaz İçi Mod	Kamera → Cihaz içi MediaPipe Hand Landmarker → 84 Koordinat → `tflite_flutter` (`model.tflite` 50 KB) → Harf Tahmini.	**İnternet gerekmez (Offline çalışır).** Sunucuya fotoğraf gönderilmez, telefon ısınmaz, gecikme < 5 ms'dir.
Mobil Hibrit / Veri Toplama Modu	Kullanıcı "*Veri Topla*" butonuna bastığında kareler `Isolate` (arka plan thread) içinde sıkıştırılıp WebSocket ile sunucuya kaydedilir.	Arayüzde donma olmaz, yeni harf verileri doğrudan telefondan toplanabilir.
Web Uygulaması (Tarayıcı)	Tarayıcı kamerası → MediaPipe Web (veya JPEG frame) → TFLite Web / Python Socket API → Sonuç Gösterimi.	Kurulum gerektirmeden hem bilgisayar hem telefon tarayıcısından linke tıklayarak çalışır.

6. Rapor 3: Canlıya Alma (Deployment) Özeti

Hedef	Uygulama Adımı	Platform / Maliyet
Mobil Uygulama (APK / AAB)	`assets/model.tflite` (50 KB) ve `labels.txt` eklenerek `flutter build apk --release` ile	Android Cihazlar / Google Play Store (Sıfır sunucu maliyeti)

Hedef	Uygulama Adımı	Platform / Maliyet
	derlenir.	
Web Uygulaması	`flutter build web --release` çıktısı alınır; HTTPS üzerinden kamera erişimi sağlanır.	Vercel, Netlify veya Firebase Hosting (Ücretsiz)
Python Backend API	`Dockerfile` + `gunicorn (eventlet)` + `opencv-python-headless` ile paketlenir.	Render.com / Railway.app (Ücretsiz / Başlangıç paketi yeterli)